

Chapter 6. Authentication, Authorization, and Admission Control

The API server is the gateway to the Kubernetes cluster. Any human user, client (e.g., `kubectl`), cluster component, or service account will access the API server by making a RESTful API call via HTTPS. It is *the* central point for performing operations like creating a Pod or deleting a Service.

In this chapter, we'll focus on the security-specific aspects relevant to the API server. For a detailed discussion on the inner workings of the API server and use of the Kubernetes API, refer to *Managing Kubernetes* by Brendan Burns and Craig Tracey (O'Reilly, 2018).

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Manage role-based access control (RBAC)
-

Processing an API Request

[Figure 6-1](#) illustrates the stages a request goes through when a call is made to the API server. For reference, you can find more information in the [Kubernetes documentation](#).

The first stage of request processing is *authentication*. Authentication validates the identity of the caller by inspecting the client certificates or bearer tokens. If the bearer token is associated with a service account, then it will be verified here.

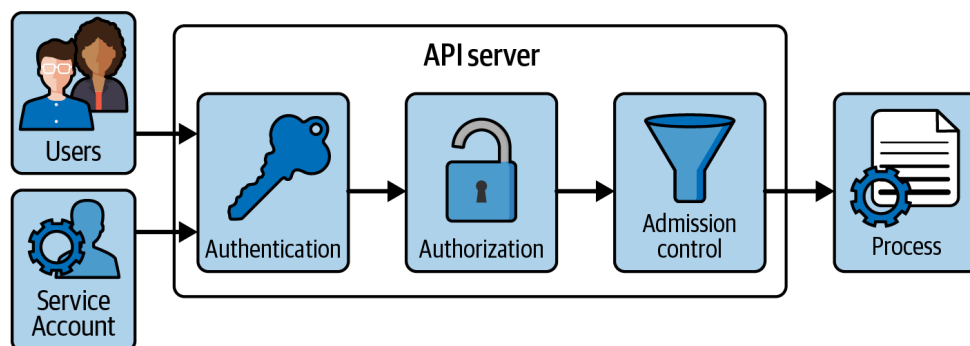


Figure 6-1. API server request processing

The second stage determines if the identity provided in the first stage can access the verb and HTTP path request. Therefore, stage two deals with *authorization* of the request, which is implemented with the standard Kubernetes RBAC model. Here, we ensure that the service account is allowed to list Pods or create a new Service object if that has been requested.

The third stage of request processing deals with *admission control*. Admission control verifies whether the request is well formed or potentially needs to be modified before the request is processed. An admission control policy could, for example, ensure that the request for creating a Pod includes the definition of a specific label. If the request doesn't define the label, then it is rejected.

Authentication with kubectl

Developers interact with the Kubernetes API by running the `kubectl` command-line tool. Whenever you execute a command with `kubectl`, the underlying HTTPS call to the API server needs to authenticate.

The Kubeconfig

Credentials for the use of `kubectl` are stored in the file `$HOME/.kube/config`, also known as the *kubeconfig file*. The kubeconfig file defines the API server endpoints of the clusters we want to interact with, as well as a list of users registered with the cluster, including their credentials in the form of client certificates. The mapping between a cluster and user for a given namespace is called a *context*. `kubectl` uses the currently selected context to know which cluster to talk to and which credentials to use.

MERGING MULTIPLE KUBECONFIG FILES

You can point the environment variable `KUBECONFIG` to a set of kubeconfig files. At runtime, `kubectl` will merge the contents of the set of defined kubeconfig files and use them. By default, `KUBECONFIG` is not set and falls back to `$HOME/.kube/config`.

[Example 6-1](#) shows a kubeconfig file. Be aware that file paths assigned in the example are user-specific and may differ in your own environment. You can find a detailed description of all configurable attributes in the [config resource type](#) API documentation.

Example 6-1. A kubeconfig file

```
apiVersion: v1
kind: Config
clusters: ❶
- cluster:
    certificate-authority: /Users/bmuschko/.minikube/ca.crt
    extensions:
    - extension:
        last-update: Mon, 09 Oct 2023 07:33:01 MDT
        provider: minikube.sigs.k8s.io
        version: v1.30.1
        name: cluster_info
        server: https://127.0.0.1:63709
    name: minikube
contexts: ❷
- context:
    cluster: minikube
    user: bmuschko
    name: bmuschko
- context:
    cluster: minikube
    extensions:
    - extension:
        last-update: Mon, 09 Oct 2023 07:33:01 MDT
        provider: minikube.sigs.k8s.io
        version: v1.30.1
        name: context_info
        namespace: default
        user: minikube
    name: minikube
current-context: minikube ❸
preferences: {}
users: ❹
- name: bmuschko
  user:
    client-key-data: <REDACTED>
- name: minikube
  user:
    client-certificate: /Users/bmuschko/.minikube/profiles/minikube/client
    client-key: /Users/bmuschko/.minikube/profiles/minikube/client.key
```

❶ A list of referential names to clusters and their API server endpoints

❷ A list of referential names to contexts (a combination of cluster and user)

❸ The currently selected context

❹ A list of referential names to users and their credentials

User management is handled by the cluster administrator. The administrator creates a user representing the developer and hands the relevant information (username and credentials) to the human wanting to interact with the cluster via `kubectl`. Alternatively, it is also possible to integrate with external identity providers for authentication purposes, e.g., via [OpenID Connect](#).

Creating a new user manually consists of multiple steps, as described in the [Kubernetes documentation](#). The developer would then add the user to the kubeconfig file on the machine intended to interact with the cluster.

Managing Kubeconfig Using `kubectl`

You do not have to manually edit the kubeconfig file(s) to change or add configuration. `kubectl` provides commands for reading and modifying its contents. The following commands provide an overview. You can find additional examples for commands in the [kubectl cheat sheet](#).

To view the merged contents of the kubeconfig file(s), run the following command:

```
$ kubectl config view
apiVersion: v1
kind: Config
clusters:
...
```

To render the currently selected context, use the `current-context` subcommand. The context named `minikube` is the active one:

```
$ kubectl config current-context
minikube
```

To change the context, provide the name with the `use-context` subcommand. Here, we are switching to the context `bmuschko`:

```
$ kubectl config use-context bmuschko
Switched to context "bmuschko".
```

To register a user with the kubeconfig file(s), use the `set-credentials` subcommand. We are choosing to assign the username `myuser` and point to the client certificate by providing the corresponding CLI flags:

```
$ kubectl config set-credentials myuser \
  --client-key=myuser.key --client-certificate=myuser.crt \
```

For the exam, familiarize yourself with the `kubectl config` command. Every task in the exam will require you to work with a specific context and/or namespace.

Authorization with Role-Based Access Control

We've learned that the API server will try to authenticate any request sent using `kubectl` by verifying the provided credentials. An authenticated request will then need to be checked against the permissions assigned to the requestor. The authorization phase of the API processing workflow checks if the operation is permitted against the requested API resource.

In Kubernetes, those permissions can be controlled using role-based access control (RBAC). In a nutshell, RBAC defines policies for users, groups, and service accounts by allowing or disallowing access to manage API resources. Enabling and configuring RBAC is mandatory for any organization with an emphasis on security.

Setting permissions is the responsibility of a cluster administrator. The following sections briefly cover the effects of RBAC on requests from users and service accounts.

RBAC Overview

RBAC helps with implementing a variety of use cases:

- Establishing a system for users with different roles to access a set of Kubernetes resources
- Controlling processes (associated with a service account) running in a Pod and performing operations against the Kubernetes API
- Limiting the visibility of certain resources per namespace

RBAC consists of three key building blocks, as shown in [Figure 6-2](#). Together, they connect API primitives and their allowed operations to the *subject*, which is a user, a group, or a service account.

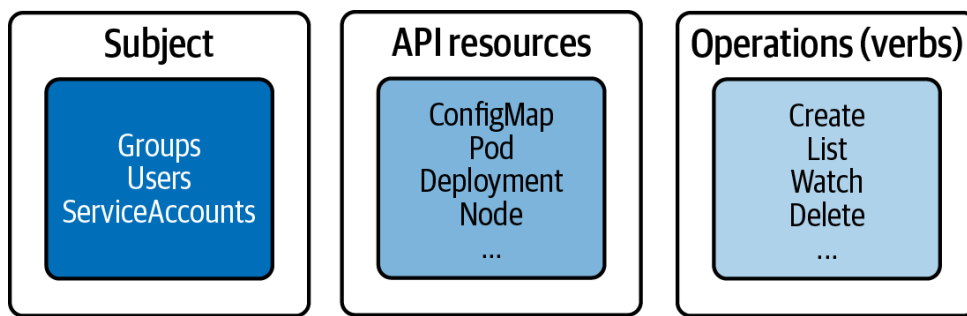


Figure 6-2. RBAC key building blocks

Each block’s responsibilities are as follows:

Subject

The user or service account that wants to access a resource

Resource

The Kubernetes API resource type (e.g., a Deployment or node)

Verb

The operation that can be executed on the resource (e.g., creating a Pod or deleting a Service)

When you need to quickly determine which operations (verbs) are supported for a specific Kubernetes resource during the exam, `kubectl api-resources -o wide` is an invaluable command that displays all available API resources along with their supported verbs (like `get`, `list`, `create`, `update`, `patch`, `watch`, `delete`).

Understanding RBAC API Primitives

With these key concepts in mind, let’s look at the Kubernetes API primitives that implement the RBAC functionality:

Role

The Role API primitive declares which of the API resources and their operations this rule should operate on in a specific namespace. For example, you may want to say “allow listing and deleting of Pods,” or you may express “allow watching the logs of Pods,” or even both with the same Role. Any operation that is not spelled out explicitly is disallowed as soon as it is bound to the subject.

RoleBinding

The RoleBinding API primitive *binds* the Role object to the subject(s) in a specific namespace. It is the glue for making the rules active. For example, you may want to say “bind the Role that permits updating Services to the user John Doe.”

[Figure 6-3](#) shows the relationship between the involved API primitives. Keep in mind that the image renders only a selected list of API resource types and operations.

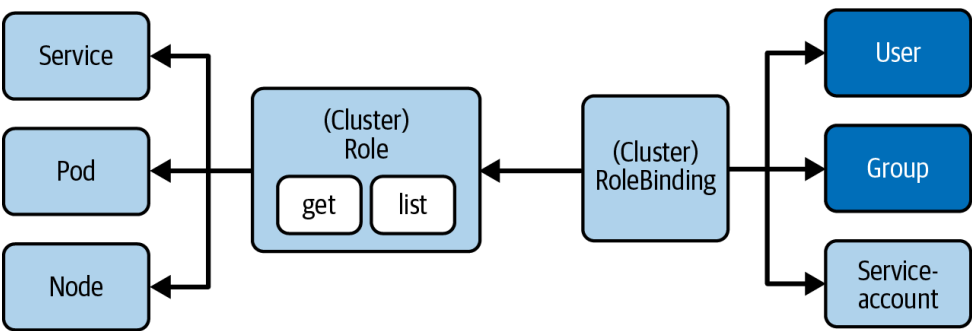


Figure 6-3. RBAC primitives

The following sections demonstrate the namespace-wide usage of Roles and RoleBindings, but the same operations and attributes apply to cluster-wide Roles and RoleBindings, discussed in [“Namespace-Wide and Cluster-Wide RBAC”](#).

Default User-Facing Roles

Kubernetes defines a set of default Roles. You can assign them to a subject via a RoleBinding or define your own custom Roles depending on your needs. [Table 6-1](#) describes the default user-facing Roles.

Table 6-1. Default user-facing Roles

Default ClusterRole	Description
cluster-admin	Allows read and write access to resources across all namespaces.
admin	Allows read and write access to resources in namespace including Roles and RoleBindings.
edit	Allows read and write access to resources in namespace except Roles and RoleBindings. Provides access to Secrets.
view	Allows read-only access to resources in namespace except Roles, RoleBindings, and Secrets.

To define new Roles and RoleBindings, you will have to use a context that allows for creating or modifying them, that is, `cluster-admin` or `admin`.

Creating Roles

Roles can be created imperatively with the `create role` command. The most important options for the command are `--verb` for defining the verbs, aka operations, and `--resource` for declaring a list of API resources (core primitives as well as CRDs). The following command creates a new Role for the resources Pod, Deployment, and Service with the verbs `list`, `get`, and `watch`:

```
$ kubectl create role read-only --verb=list,get,watch \
  --resource=pods,deployments,services
role.rbac.authorization.k8s.io/read-only created
```

Declaring multiple verbs and resources for a single imperative `create role` command can be declared as a comma-separated list for the corresponding command-line option or as multiple arguments. For example, `--verb=list,get,watch` and `--verb=list --verb=get --verb=watch` carry the same instructions. You also can use the wildcard `*` to refer to all verbs or resources.

The command-line option `--resource-name` spells out one or many object names that the policy rules should apply to. A name of a Pod could be `nginx` and listed here with its name. Providing a list of resource names is optional. If no names have been provided, then the provided rules apply to all objects of a resource type.

The declarative approach can become a little lengthy. As you can see in [Example 6-2](#), the section `rules` lists the resources and verbs. Resources with an API group, like Deployments that use the API version `apps/v1`, need to explicitly declare it under the attribute `apiGroups`. All other resources (e.g., Pods and Services) simply use an empty string, as their API version doesn't contain a group. Be aware that the imperative command for creating a Role automatically determines the API group.

Example 6-2. A YAML manifest defining a Role

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: read-only
rules:
- apiGroups:
  - ""
  resources:
  - pods
  - services
  verbs:
```


- list
- get
- watch
- **apiGroups:** **❶**
 - apps
- resources:**
 - deployments
- verbs:**
 - list
 - get
 - watch

❶ Any resource that belongs to an API group needs to be listed as an explicit rule in addition to the API resources that do not belong to an API group.

Listing Roles

Once the Role has been created, its object can be listed. The list of Roles renders only the name and the creation timestamp. Each of the listed roles does not give away any of its details:

```
$ kubectl get roles
NAME          CREATED AT
read-only     2021-06-23T19:46:48Z
```

Rendering Role Details

You can inspect the details of a Role using the `describe` command. The output renders a table that maps a resource to its permitted verbs:

```
$ kubectl describe role read-only
Name:          read-only
Labels:        <none>
Annotations:   <none>
PolicyRule:
  Resources          Non-Resource URLs  Resource Names      Verbs
  -----
  pods               [ ]                [ ]                 [list get watch]
  services           [ ]                [ ]                 [list get watch]
  deployments.apps   [ ]                [ ]                 [list get watch]
```

This cluster has no resources created, so the list of resource names in the following console output is currently empty.

Creating RoleBindings

The imperative command to create a RoleBinding object is `create rolebinding`. To bind a Role to the RoleBinding, use the `--role` command-line option. The subject type can be assigned by declaring the options `--user`, `--group`, or `--serviceaccount`. The following command creates the RoleBinding with the name `read-only-binding` to the user called `bmuschko`:

```
$ kubectl create rolebinding read-only-binding --role=read-only --user=bmuschko
rolebinding.rbac.authorization.k8s.io/read-only-binding created
```

[Example 6-3](#) shows a YAML manifest representing the RoleBinding. You can see from the structure that a role can be mapped to one or many subjects. The data type is an array indicated by the dash character under the attribute `subjects`. At this time, only the user `bmuschko` has been assigned.

Example 6-3. A YAML manifest defining a RoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-only-binding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: read-only
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: bmuschko
```

Listing RoleBindings

The most important information the list of RoleBindings displays is the associated Role. The following command shows that the RoleBinding `read-only-binding` has been mapped to the Role `read-only`:

```
$ kubectl get rolebindings
```

NAME	ROLE	AGE
read-only-binding	Role/read-only	24h

The output does not provide an indication of the subjects. You will need to render the details of the object for more information, as described in the next section.

Rendering RoleBinding Details

RoleBindings can be inspected using the `describe` command. The output renders a table of subjects and the assigned role. The following example renders the descriptive representation of the RoleBinding named `read-only-binding`:

```
$ kubectl describe rolebinding read-only-binding
Name:          read-only-binding
Labels:        <none>
Annotations:   <none>
Role:
  Kind:  Role
  Name:  read-only
Subjects:
  Kind  Name      Namespace
  ----  -
  User  bmuschko
```

Seeing the RBAC Rules in Effect

Let's see how Kubernetes enforces the RBAC rules for the scenario we've set up so far. First, we'll create a new Deployment with the `cluster-admin` permissions. In minikube, those permissions are available to the context `minikube` by default:

```
$ kubectl config current-context
minikube
$ kubectl create deployment myapp --image=nginx:1.25.2 --port=80 --replica
deployment.apps/myapp created
```

Now, we'll switch the context for the user `bmuschko`:

```
$ kubectl config use-context bmuschko-context
Switched to context "bmuschko-context".
```

Remember that the user `bmuschko` is permitted to list Deployments. We'll verify that by using the `get deployments` command:

```
$ kubectl get deployments
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
myapp     2/2     2            2           8s
```

The RBAC rules allow listing Deployments, Pods, and Services only. The following command tries to list the ReplicaSets, which results in an error:

```
$ kubectl get replicaset
```

```
Error from server (Forbidden): replicaset.apps is forbidden: User "bmuschko" cannot list resource "replicaset" in API group "apps" in the namespace "default"
```

A similar behavior can be observed when trying to use verbs other than `list`, `get`, or `watch`. The following command tries to delete a Deployment:

```
$ kubectl delete deployment myapp
```

```
Error from server (Forbidden): deployment.apps "myapp" is forbidden: User "bmuschko" cannot delete resource "deployment" in API group "apps" in the namespace "default"
```

At any given time, you can check a user's permissions with the `auth can-i` command. The command gives you the option to list all permissions or check a specific permission:

```
$ kubectl auth can-i --list --as bmuschko
```

Resources	Non-Resource URLs	Resource Names	Verbs
...			
pods	[]	[]	[list get watch]
services	[]	[]	[list get watch]
deployments.apps	[]	[]	[list get watch]

```
$ kubectl auth can-i list pods --as bmuschko
```

```
yes
```

Namespace-Wide and Cluster-Wide RBAC

Roles and RoleBindings apply to a particular namespace. You will have to specify the namespace when creating both objects. Sometimes, a set of Roles and RoleBindings needs to apply to multiple namespaces or even to the whole cluster. For a cluster-wide definition, Kubernetes offers the API resource types `ClusterRole` and `ClusterRoleBinding`. The configuration elements are effectively the same. The only difference is the value of the `kind` attribute:

- To define a cluster-wide Role, use the imperative subcommand `clusterrole` or the kind `ClusterRole` in the YAML manifest.
- To define a cluster-wide RoleBinding, use the imperative subcommand `clusterrolebinding` or the kind `ClusterRoleBinding` in the YAML manifest.

ClusterRoles and ClusterRoleBindings not only set up cluster-wide permissions to a namespaced resource, but they can also be used to set up permissions for non-namespaced resources like CRDs and nodes.

Aggregating RBAC Rules

Existing ClusterRoles can be aggregated to avoid having to redefine a new, composed set of rules that likely leads to duplication of instructions. For example, say you wanted to combine a user-facing role with a custom Role. An aggregated ClusterRole can merge rules via label selection without having to copy-paste the existing rules into one.

Say we defined two ClusterRoles shown in [Example 6-4](#) and [Example 6-5](#). The ClusterRole `list-pods` allows for listing Pods and the ClusterRole `delete-services` allows for deleting Services.

Example 6-4. A YAML manifest defining a ClusterRole for listing Pods

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: list-pods
  namespace: rbac-example
  labels:
    rbac-pod-list: "true"
rules:
- apiGroups:
  - ""
  resources:
  - pods
  verbs:
  - list
```

Example 6-5. A YAML manifest defining a ClusterRole for deleting Services

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: delete-services
  namespace: rbac-example
  labels:
    rbac-service-delete: "true"
rules:
- apiGroups:
  - ""
  resources:
  - services
  verbs:
  - delete
```

To aggregate those rules, ClusterRoles can specify an `aggregationRule`. This attribute describes the label selection rules. [Example 6-6](#) shows an

aggregated ClusterRole defined by an array of `matchLabels` criteria. This ClusterRole does not add its own rules as indicated by `rules: []`; however, there's no limiting factor that would disallow it.

Example 6-6. A YAML manifest defining a ClusterRole with aggregated rules

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: pods-services-aggregation-rules
  namespace: rbac-example
aggregationRule:
  clusterRoleSelectors:
  - matchLabels:
      rbac-pod-list: "true"
  - matchLabels:
      rbac-service-delete: "true"
rules: []
```

We can verify the proper aggregation behavior of the ClusterRole by describing the object. You can see in the following output that both ClusterRoles, `list-pods` and `delete-services`, have been taken into account:

```
$ kubectl describe clusterroles pods-services-aggregation-rules -n rbac-ex
Name:          pods-services-aggregation-rules
Labels:        <none>
Annotations:   <none>
PolicyRule:
  Resources  Non-Resource URLs  Resource Names  Verbs
  -----  -
  services   []                 []              [delete]
  pods       []                 []              [list]
```

For more information on ClusterRole label selection rules, see the [official documentation](#). The page also explains how to aggregate the default user-facing ClusterRoles.

Working with Service Accounts

We've been using the `kubectl` executable to run operations against a Kubernetes cluster. Under the hood, its implementation calls the API server by making an HTTP call to the exposed endpoints. Some applications running inside of a Pod may have to communicate with the API server as well. For example, the application may ask for specific cluster node information or available namespaces.

Pods can use a service account to authenticate with the API server through an authentication token. A Kubernetes administrator assigns rules to a service account via RBAC to authorize access to specific resources and actions as illustrated in [Figure 6-4](#).



Figure 6-4. Using a service account to communicate with an API server

A Pod doesn't necessarily need to be involved in the process. Other use cases call for leveraging a service account outside of a Kubernetes cluster. For example, you may want to communicate with the API server as part of a CI/CD pipeline automation step. The service account can provide the credentials to authenticate with the API server.

The Default Service Account

So far, we haven't defined a service account for a Pod. If not assigned explicitly, a Pod uses the [default service account](#), which has the same permissions as an unauthenticated user. This means that the Pod cannot view or modify the cluster state, or list or modify any of its resources. The default service account can, however, request basic cluster information via the assigned `system:discovery` Role.

You can query for the available service accounts with the subcommand `serviceaccounts`. You should see only the default service account listed in the output:

```
$ kubectl get serviceaccounts
NAME          SECRETS  AGE
default       0         4d
```

While you can execute the `kubectl` operation to delete the default service account, Kubernetes will reinstantiate the service account immediately.

Creating a Service Account

You can create a custom service account object using the imperative and declarative approach. This command creates a service account object with the name `cicd-bot`. The assumption here is to use the service account for calls to the API server made by a CI/CD pipeline:

```
$ kubectl create serviceaccount cicd-bot
```

You can also represent the service account in the form of a manifest. In its simplest form, the definition assigns the kind `ServiceAccount` and a name, as shown in [Example 6-7](#).

Example 6-7. YAML manifest for a service account

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: cicd-bot
```

You can set a couple of [configuration options](#) for a service account. For example, you may want to disable automounting of the authentication token when assigning the service account to a Pod. Although you will not need to understand those configuration options for the exam, it makes sense to dive deeper into security best practices by reading up on them in the Kubernetes documentation.

Setting Permissions for a Service Account

It's important to limit the permissions to only the service accounts that are necessary for the application to function. The next sections will explain how we will achieve this to minimize the potential attack service.

For this scenario to work, you'll need to create a `ServiceAccount` object and assign it to the Pod. Service accounts can be tied in with RBAC and assigned a `Role` and `RoleBinding` to define which operations they should be allowed to perform.

Binding the service account to a Pod

As a starting point, we will set up a Pod that lists all Pods and Deployments in the namespace `k97` by calling the Kubernetes API. The call is made as part of an infinite loop every ten seconds. The response from the API call will be written to standard output accessible via the Pod's logs.

ACCESSING THE API SERVER ENDPOINT

Accessing the Kubernetes API from a Pod is straightforward. Instead of using the IP address and port for the API server Pod, you can simply refer to a Service named `kubernetes.default.svc` instead. This special Service lives in the `default` namespace and is stood up by the cluster automatically.

To authenticate against the API server, we'll send a bearer token associated with the service account used by the Pod. The default behavior of a service account is to auto-mount API credentials on the path `/var/run/secrets/kubernetes.io/serviceaccount/token`. We'll simply get the contents of the file using the `cat` command-line tool and send them along as a header for the HTTP request. [Example 6-8](#) defines the namespace, the service account, and the Pod in a single YAML manifest file: `setup.yaml`.

Example 6-8. YAML manifest for assigning a service account to a Pod

```
apiVersion: v1
kind: Namespace
metadata:
  name: k97
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: sa-api
  namespace: k97
---
apiVersion: v1
kind: Pod
metadata:
  name: list-objects
  namespace: k97
spec:
  serviceAccountName: sa-api ❶
  containers:
  - name: pods
    image: alpine/curl:3.14
    command: ['sh', '-c', 'while true; do curl -s -k -m 5 -H \
      "Authorization: Bearer $(cat /var/run/secrets/kubernetes.io/
      serviceaccount/token)" https://kubernetes.default.svc.cluste
      local/api/v1/namespaces/k97/pods; sleep 10; done'] ❷
  - name: deployments
    image: alpine/curl:3.14
    command: ['sh', '-c', 'while true; do curl -s -k -m 5 -H \
      "Authorization: Bearer $(cat /var/run/secrets/kubernetes.io/
      serviceaccount/token)" https://kubernetes.default.svc.cluste
      local/apis/apps/v1/namespaces/k97/deployments;
      sleep 10; done'] ❸
```

❶ The service account referenced by name used for communicating with the Kubernetes API

❷ Performs an API call to retrieve the list of Pods in the namespace k97

❸

Performs an API call to retrieve the list of Deployments in the namespace k97

Create the objects from the YAML manifest with the following command:

```
$ kubectl apply -f setup.yaml
namespace/k97 created
serviceaccount/sa-api created
pod/list-objects created
```

Verifying the default permissions

The Pod named `list-objects` makes a call to the API server to retrieve the list of Pods and Deployments in dedicated containers. The container `Pods` performs the call to list Pods. The container `deployments` sends a request to the API server to list Deployments.

As explained in the [Kubernetes documentation](#), the default RBAC policies do not grant any permissions to service accounts outside of the `kube-system` namespace. The logs of the containers `Pods` and `deployments` return an error message indicating that the service account `sa-api` is not authorized to list the resources:

```
$ kubectl logs list-objects -c pods -n k97
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "pods is forbidden: User \"system:serviceaccount:k97:sa-api\"
            cannot list resource \"pods\" in API group \"\" in the \
            namespace \"k97\"",
  "reason": "Forbidden",
  "details": {
    "kind": "pods"
  },
  "code": 403
}
$ kubectl logs list-objects -c deployments -n k97
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "deployments.apps is forbidden: User \
            \"system:serviceaccount:k97:sa-api\" cannot list resource \
            \"deployments\" in API group \"apps\" in the namespace \
            \"k97\"",
  "reason": "Forbidden",
  "details": {
```

```

    "group": "apps",
    "kind": "deployments"
  },
  "code": 403
}

```

Next up, we'll stand up a Role and RoleBinding object with the required API permissions to perform the necessary calls.

Creating the Role

Start by defining the Role named `list-pods-role` shown in [Example 6-9](#) in the file `role.yaml`. The set of the rules adds only the Pod resource and the verb `list`.

Example 6-9. YAML manifest for a Role that allows listing Pods

```

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: list-pods-role
  namespace: k97
rules:
- apiGroups: [ "" ]
  resources: [ "pods" ]
  verbs: [ "list" ]

```

Create the object by pointing to its corresponding YAML manifest file:

```

$ kubectl apply -f role.yaml
role.rbac.authorization.k8s.io/list-pods-role created

```

Creating the RoleBinding

[Example 6-10](#) defines the YAML manifest for the RoleBinding in the file `rolebinding.yaml`. The RoleBinding maps the Role `list-pods-role` to the service account named `sa-pod-api` and applies it only to the namespace `k97`.

Example 6-10. YAML manifest for a RoleBinding attached to a service account

```

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: serviceaccount-pod-rolebinding
  namespace: k97

```

```

subjects:
- kind: ServiceAccount
  name: sa-api
roleRef:
  kind: Role
  name: list-pods-role
  apiGroup: rbac.authorization.k8s.io

```

Create both RoleBinding objects using the `apply` command:

```

$ kubectl apply -f rolebinding.yaml
rolebinding.rbac.authorization.k8s.io/serviceaccount-pod-rolebinding created

```

Verifying the granted permissions

With the granted `list` permissions, the service account can now properly retrieve all the Pods in the `k97` namespace. The `curl` command in the `pods` container succeeds, as shown in the following output:

```

$ kubectl logs list-objects -c pods -n k97
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "resourceVersion": "628"
  },
  "items": [
    {
      "metadata": {
        "name": "list-objects",
        "namespace": "k97",
        ...
      }
    }
  ]
}

```

We did not grant any permissions to the service account for other resources. Listing the Deployments in the `k97` namespace still fails. The following output shows the response from the `curl` command in the `deployments` namespace:

```

$ kubectl logs list-objects -c deployments -n k97
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "deployments.apps is forbidden: User \

```

```

        \"system:serviceaccount:k97:sa-api\" cannot list resource \\
        \"deployments\" in API group \\\"apps\\\" in the namespace \\
        \\\"k97\\\"\",
    \"reason\": \"Forbidden\",
    \"details\": {
        \"group\": \"apps\",
        \"kind\": \"deployments\"
    },
    \"code\": 403
}

```

Feel free to modify the Role object to allow listing Deployment objects as well.

Admission Control

The last phase of processing a request to the API server is admission control. Admission control is implemented by admission controllers. An admission controller provides a webhook to approve, deny, or mutate a request before it takes effect.

Admission controllers can be registered with the configuration of the API server. By default, the configuration file can be found at */etc/kubernetes/manifests/kube-apiserver.yaml*. It is the cluster administrator's job to manage the API server configuration. The following command-line invocation of the API server enables the [admission control plug-ins](#) named `NamespaceLifecycle`, `PodSecurity`, and `LimitRanger`:

```
$ kube-apiserver --enable-admission-plugins=NamespaceLifecycle,PodSecurity,LimitRanger
```

Developers will inadvertently use admission control plug-ins that have been configured by the administrator. Two examples are the [LimitRanger](#) and the [ResourceQuota](#), which I'll discuss in [“Working with Limit Ranges”](#) and [“Working with Resource Quotas”](#).

Summary

The API server processes requests to the Kubernetes API. Every request has to go through three phases: authentication, authorization, and admission control. Every phase can prevent further processing. For example, if the credentials sent with the request cannot be authenticated, then the request will be dropped.

We looked at examples of all phases. The authentication phase covered `kubectl` as the client making a call to the Kubernetes API. The `kubeconfig` file serves as configuration source for named clusters, users, and their credentials. In Kubernetes, authorization is handled by RBAC. We learned the Kubernetes primitives that let you configure permissions for API resources tied to one or many subjects.

Finally, we briefly examined the purpose of admission control and listed some plug-ins that act as controllers for validating or mutating a request to the Kubernetes API.

Exam Essentials

Practice interacting with the Kubernetes API

This chapter demonstrated some ways to communicate with the Kubernetes API. We performed API requests by switching to a user context and with the help of a RESTful API call using `curl`. Explore the [Kubernetes API](#) and its endpoints on your own for broader exposure.

Understand the implications of defining RBAC rules for users and service accounts

Anonymous user requests to the Kubernetes API will not allow any substantial operations. For requests coming from a user or a service account, you will need to carefully analyze permissions granted to the subject. Learn the ins and outs of defining RBAC rules by creating the relevant objects to control permissions. Service accounts automount a token when used in a Pod. Expose the token as a volume only if you are intending to make API calls from the Pod.

Be aware of the purpose of admission control

The API server comes with preconfigured admission control plug-ins that support the functionality of Kubernetes primitives like `LimitRange`. For the exam, you will not have to have a deep understanding of enabling or configuring admission control plug-ins.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. Navigate to the directory `app-a/ch06/rbac-aggregation` of the checked-out GitHub repository [bmuschko/cka-study-guide](#).

Create a ClusterRole with the name `service-view` to the API resources `services` with the operations `get` and `list`. Create the RoleBinding named `ellasmith-service-view` in the `development` namespace. Map the user `ellasmith` to the ClusterRole `service-view`.

Create a ClusterRole with the name `combined`. Aggregate cluster roles based on the matching label key-value pair

`rbac.cka.cncf.com/aggregate: "true"`. Render the selected rules of the ClusterRole `combined`. How many rules do you see?

Create a ClusterRole with the name `deployment-modify` to the API resources `deployments` with the operations `create`, `delete`, `patch`, and `update`. Assign the label key-value pair

`rbac.cka.cncf.com/aggregate: "true"`. Render the selected rules of the ClusterRole `combined`. How many rules do you see?

Run a command to figure out if the user `ellasmith` can list Services in the namespace `development`. Write the output of the command to the file `list-services-ellasmith.txt`. The output is either `no` or `yes`.

Run a command to figure out if the user `ellasmith` can watch Deployments in the namespace `production`. Write the output of the command to the file `watch-deployments-ellasmith.txt`. The output is either `no` or `yes`.

Prerequisite: This exercise requires the installation of the tools [Vagrant](#) and a [VMware provider](#).

2. Create the ServiceAccount named `api-access` in a new namespace called `apps`.

Create a ClusterRole with the name `api-clusterrole`, and create a ClusterRoleBinding named `api-clusterrolebinding`. Map the ServiceAccount from the previous step to the API resources `Pods` with the operations `watch`, `list`, and `get`.

Create a Pod named `operator` with the image `nginx:1.21.1` in the namespace `apps`. Expose the container port 80. Assign the ServiceAccount `api-access` to the Pod. Create another Pod named `disposable` with the image `nginx:1.21.1` in the namespace `rm`. Do not assign the ServiceAccount to the Pod.

Open an interactive shell to the Pod named `operator`. Use the command-line tool `curl` to make an API call to list the Pods in the namespace `rm`. What response do you expect? Use the command-line tool `curl` to make an API call to delete the Pod `disposable` in the namespace `rm`. Does the response differ from the first call? You can find information about how to interact with Pods using the API via HTTP in the [Kubernetes documentation](#).